arXiv:2505.07773v4 [cs.AI] 20 Aug 2025

Agent RL Scaling Law: Spontaneous Code Execution for Mathematical Problem Solving

Xinji Mai[1,2†] **Haotian Xu**[2†] **Zhong-Zhi Li**[5] **Xing W**[2] **Weinong Wang**[2] **Jian Hu**[4]
Yingying Zhang[3*] **Wenqiang Zhang**[1*]
[1]Fudan University
[2]Xiaohongshu
[3]East China Normal University
[4]Independent Researcher
[5]University of Chinese Academy of Sciences
xjmai23@m.fudan.edu.cn,{xuhaotian,wuxing,wangweinong}@xiaohongshu.com
lizhongzhi2022@ia.ac.cn, janhu9527@gmail.com

Abstract

Large Language Models (LLMs) often struggle with mathematical reasoning tasks requiring precise, verifiable computation. While Reinforcement Learning (RL) from outcome-based rewards enhances text-based reasoning, understanding how agents autonomously learn to leverage external tools like code execution remains crucial. We investigate RL from outcome-based rewards for Tool-Integrated Reasoning, ZeroTIR, training base LLMs to spontaneously generate and execute Python code for mathematical problems without supervised tool-use examples. Our central contribution is we demonstrate that as RL training progresses, key metrics scale predictably. Specifically, we observe strong positive correlations where increased training steps lead to increases in the spontaneous code execution frequency, the average response length, and, critically, the final task accuracy. This suggests a quantifiable relationship between computational effort invested in training and the emergence of effective, tool-augmented reasoning strategies. We implement a robust framework featuring a decoupled code execution environment and validate our findings across standard RL algorithms and frameworks. Experiments show ZeroTIR significantly surpasses non-tool ZeroRL baselines on challenging math benchmarks. Our findings provide a foundational understanding of how autonomous tool use is acquired and scales within Agent RL, offering a reproducible benchmark for future studies. Code is released at https://github.com/yyht/openrlhf_async_pipline.

1 Introduction

LLMs have demonstrated remarkable capabilities across various domains. However, they often face challenges when tackling tasks that demand precise, multi-step reasoning and complex computations, particularly within the realm of mathematicsWang et al. [2023], Gao et al. [2022a]. The inherent nature of next-token prediction often leads LLMs to generate responses based on high likelihood rather than computational correctness, hindering their reliability for mathematical problem-solving. Existing approaches to augment the mathematical abilities of LLMs typically involve Supervised Fine-Tuning (SFT) on specific datasets or integrating external tools in a controlled mannerWang et al. [2024]. SFT, while potentially effective, often necessitates extensive, high-quality trajectory data and may constrain the model's capacity to explore novel problem-solving strategies, potentially leading to overfitting on specific solution patterns and sacrificing generalizability. Tool-Integrated Reasoning (TIR) methods, such as the TIR capability mentioned in the context of Qwen2.5-Math Yang et al.

† Equal contribution. * Corresponding authors.

Preprint. Under review.

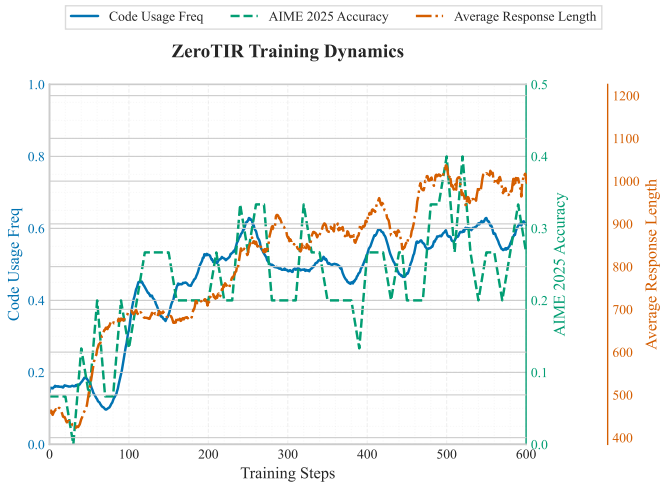


Figure 1: Illustration of Agent RL Scaling Law during Zero-TIR training. The observed trends, including the non-monotonic scaling of code usage and the positive correlation between training steps, code usage, response length, and task accuracy, exemplify the core scaling dynamics.

[2024], usually depend on specific prompt structures or predefined triggers to invoke tools like code interpreters. This paradigm differs fundamentally from fostering an agent that learns to utilize tools *spontaneously* based on the emergent needs of the task.

The advent of models like DeepSeek-R1 Guo et al. [2025] has highlighted the significant success of RL in scaling LLM reasoning capabilities directly from base models, which is also known as ZeroRL. This approach enables emergent abilities, such as self-correction and reflection, using only outcome-based rewards. Works like Open DeepResearch Roucher et al. [2025] have underscored the immense potential of tool invocation within LLMs. However, the application of agentic tool use, particularly code execution for mathematical tasks, has received comparatively less attention. We posit that for mathematical problems, precise and deterministic computation via code execution is often more advantageous than retrieving potentially noisy textual information from search engines, primarily due to the deterministic nature and rapid feedback loop of code execution environmentsGao et al. [2022b], Chen et al. [2023]. We also note contemporaneous work, such as the excellent contribution by TORL Li et al. [2025], which explores the potential of using code execution for mathematical tasks.

However, we believe that RL from a fine-tuned model with existing tooling capabilities obscures some important findings. Similar to RL from a model after SFT, it is difficult to observe a relationship between response length and performance. The paper aims to provide a more comprehensive and clearer analysis to facilitate community research and reproduction of what we term Agent RL Scaling Law. We present exhaustive experiments conducted using mainstream community frameworks (Open-Reasoner-Zero Hu et al. [2025b], OpenRLHF Sheng et al. [2024]) and popular RL algorithms (PPO Schulman et al. [2017], Reinforce++ Hu [2025]), coupled with an environment server. We investigate how LLMs, initialized from base models, can spontaneously learn to leverage a Python code execution environment through RL. Our central hypothesis is that the learning process for utilizing such a tool adheres to discernible patterns, which we designate as Agent RL Scaling Law.

- We identify and characterize novel Agent RL Scaling Law that govern the autonomous acquisition of spontaneous code execution skills in ZeroTIR for mathematical reasoning.
- We propose and implement an effective framework, ARL, for training base LLMs to spontaneously leverage code execution, which can be quickly enabled on the community mainstream RL training frameworks.
- We provide extensive empirical validation showing that ZTRL model trained with ZeroTIR significantly outperform non-instrumental ZeroRL baselines on challenging mathematical benchmarks and SFT-based TIR methods.

By discovering the Agent RL Scaling Law and developing a robust framework to leverage them, our approach significantly advances the understanding and application of reinforcement learning

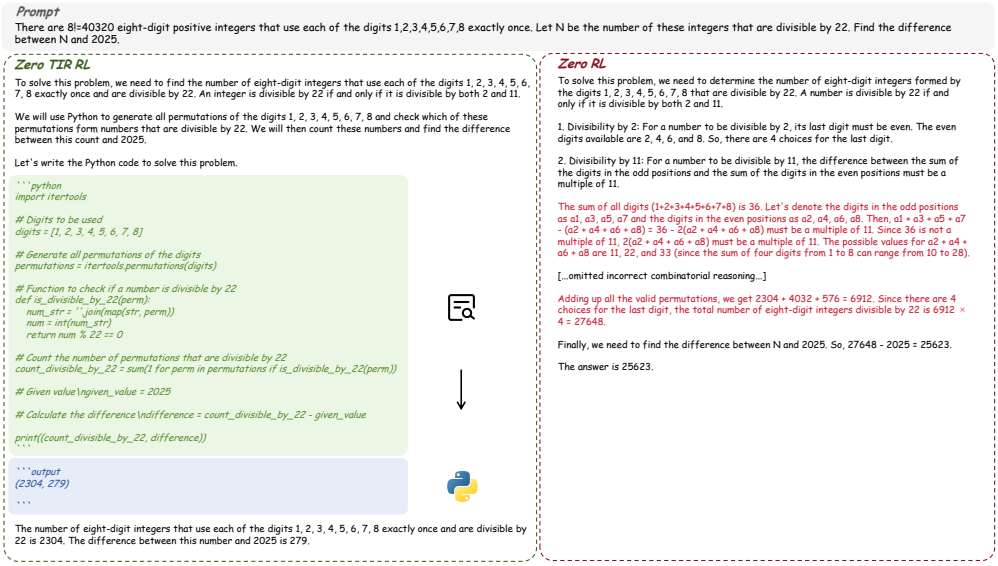


Figure 2: Example responses to the same prompt, contrasting the code-augmented solution generated by ZeroTIR (left) with the reasoning-only solution from ZeroRL (right).

for training capable LLM agents that autonomously utilize tools for complex reasoning tasks like mathematics.

2 Related Works

Our research builds upon advancements in LLM reasoning, TIR, and agent reinforcement learning.

2.1 Tool-Integrated Reasoning

To overcome the limitations of purely internal reasoning, particularly in domains requiring calculation or factual look-upWei et al. [2022], Hou et al. [2024], TIR approaches augment LLMs with external tools. These tools can range from calculators and search engines to code interpreters. Much existing work implements TIR through SFT, training models on datasets containing tool invocation examples, or through carefully designed prompts and controlled mechanisms that trigger tool use under specific conditions, such as Qwen2.5-Math-TIR Yang et al. [2024], Yao et al. [2023]). These methods typically guide or compel the model to use tools according to predefined patterns or explicit instructionsSchick et al. [2023]. Our research diverges from these controlled approaches by focusing on spontaneous tool use. We investigate whether an LLM agent can learn autonomously, through ZeroRL, when and how to utilize a code execution tool based purely on maximizing task success reward, without being explicitly instructed or pre-trained on tool-use trajectories.

2.2 Agent Reinforcement Learning

LLM Agents, capable of autonomous planning, decision-making, and environmental interaction including tool use, represent a significant evolution Qu et al. [2025]. Reinforcement Learning provides a powerful paradigm for training these agents. Agent RL has been applied to train LLMs for information retrieval tasks, with frameworks like Search-R1 Jin et al. [2025] and R1-Searcher Sun et al. [2025b] teaching models to autonomously query search engines during reasoning using outcome-based rewardsNakano et al. [2022]. This principle is also being applied to computational tools; Contemporaneous work like TORL Li et al. [2025] uses ZeroRL to train agents for code interpreter use in mathematics. A notable trend in Agent RL is the effectiveness of simpler, outcome-based rewards over complex process rewards or imitation learning, fostering exploration and the emergence of novel strategies. We situate our research within the Agent RL landscape, specifically focusing on the integration of a code execution tool for enhancing mathematical reasoning of base model. Distinct from search-focused agents and building upon the ZeroRL paradigm, our primary contribution lies in the systematic identification and analysis of the Agent RL Scaling Law that govern

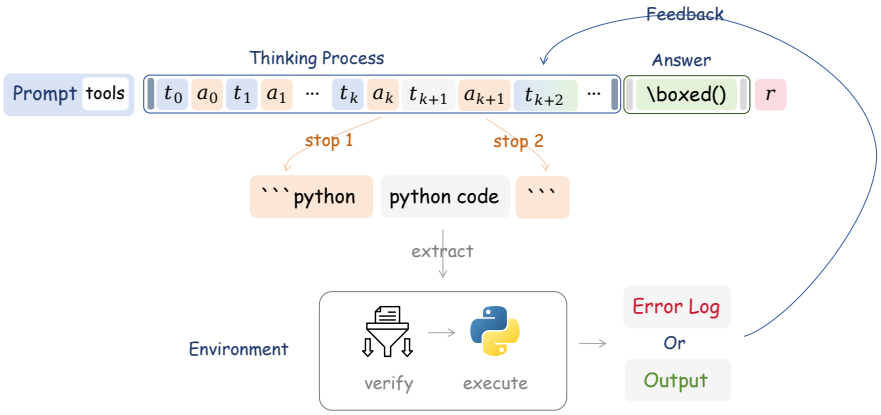


Figure 3: Detailed schematic of the interactive rollout process.

the learning dynamics of spontaneous tool acquisition across various RL algorithms and frameworks, providing a foundational understanding and a reproducible benchmark for this specific type of Agent RL.

3 Methodology

ZeroTIR, trains a base LLM to autonomously utilize a Python code execution environment for mathematical problem-solving through reinforcement learning. This section details the core components of our framework, including the RL formulation, the interaction mechanism between the LLM and the code environment, and specific design choices made during implementation and training.

3.1 RL for Spontaneous Code Execution

Our methodology utilizes reinforcement learning to train the LLM agent, primarily employing policy gradient algorithms such as PPO and REINFORCE variants like Reinforce++. PPO operates as an actor-critic method, optimizing a policy network π_θ while concurrently training a value network V_ϕ to estimate state values, which aids in reducing the variance of policy gradient estimates. The policy update in PPO typically maximizes a clipped surrogate objective function:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the probability ratio between the current policy and the policy used to collect the data ($\pi_{\theta_{old}}$), $\hat{A}_t$ is the estimated advantage at timestep t, and ϵ is a hyperparameter defining the clipping range. The value function V_ϕ is trained concurrently, usually by minimizing the mean squared error against target values derived from sampled rewards.

A crucial aspect of applying PPO in our setting involves handling the tokens inserted by the external code execution environment $\mathcal{E}_{code}$. The value function $V_\phi(s_t)$ should not be trained on states s_t corresponding to these environment-generated tokens, as they are not products of the policy π_θ. Thus, the value loss computation must mask these states. Furthermore, the advantage estimation $\hat{A}_t$, especially when using Generalized Advantage Estimation (GAE) with $\lambda \neq 1$, requires careful consideration. GAE computes advantages based on temporal difference (TD) errors:

$$\hat{A}_t^{GAE} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \text{where} \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Here, γ is the discount factor and λ is the GAE parameter balancing bias and variance. Since $V_\phi(s_{t+1})$ is unreliable or ill-defined if s_{t+1} consists of environment feedback tokens, terms involving these states can introduce noise or bias into $\hat{A}_t^{GAE}$. A common practice is to effectively mask the influence of these tokens, potentially by setting $\hat{A}_t = 0$ at positions corresponding to environment-inserted

tokens during the policy loss calculation, preventing spurious updates to π_θ. However, we found that setting λ to 1 did not affect the results, so here we set λ to 1.

REINFORCE-style algorithms, including Reinforce++, operate differently, typically foregoing an explicit learned value function. The policy gradient is estimated directly using sampled trajectories. The general form of the policy gradient objective is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \hat{A}_t \right]$$

where the agent learns by adjusting θ in the direction suggested by the gradient. The advantage estimate $\hat{A}_t$ in this context is often calculated using the discounted Monte Carlo return G_t minus a baseline $b(s_t)$:

$$\hat{A}_t = G_t - b(s_t), \quad \text{where} \quad G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$$

The baseline $b(s_t)$ serves to reduce variance and can be a simple running average of returns or potentially more sophisticated estimates, but crucially, it doesn't rely on a learned value function V_ϕ in the same way PPO does. While the advantage calculation itself is less directly complicated by the value of environment states, careful reward attribution and ensuring only policy-generated actions (a_t corresponding to LLM tokens) contribute to the gradient term $\nabla_\theta \log \pi_\theta(a_t|s_t)$ remain essential. Reinforce++ specifically may incorporate advanced baseline techniques or modifications to the update rule for improved performance compared to vanilla REINFORCE.

Irrespective of the specific algorithm, the overall learning objective remains the maximization of the expected outcome-based reward $R(x, y)$, regularized by the KL divergence from a reference policy π_{ref} to maintain stability:

$$\max_\theta \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x; \mathcal{E}_{code})} [R(x, y)] - \beta D_{KL}[\pi_\theta(\cdot|x; \mathcal{E}_{code}) || \pi_{ref}(\cdot|x; \mathcal{E}_{code})]$$

Here, x is the input problem, y is the full trajectory including interactions with the code environment $\mathcal{E}_{code}$, $R(x, y)$ is the outcome-based reward, π_{ref} is the reference policy, D_{KL} is the KL divergence, and β controls regularization.

3.2 Training Stability and Efficiency Techniques

Training stability and efficiency are crucial in interactive RL settings like ZeroTIR. To address potential instability, such as training collapse observed in some ZeroRL frameworks, we employ two key techniques.

First, we introduce a replay buffer filtering mechanism to enhance stability and focus learning. Multiple responses generated for the same prompt are grouped, and their final answer accuracy (based on outcome rewards) is calculated. We filter out groups with accuracy above a high threshold 0.8 or below a low threshold 0.2, prioritizing samples within the intermediate range where the learning gradient is likely most beneficial.

Second, we implement an efficient interaction mechanism for spontaneous code execution during rollouts, depicted schematically in Figure 3 and detailed in Algorithm 1. This method leverages dynamic stop tokens (e.g., "‘python, "‘) to iteratively manage reasoning, code generation, interaction with the external code environment, and integration of execution feedback. This state-machine approach is significantly more efficient than generating complete sequences followed by post-hoc parsing for code extraction. This mechanism also facilitates managing tool interaction frequency by counting completed execution cycles (n_{calls}). For experimental control, particularly in initial runs managing computational resources, we enforce a maximum call limit (N_{max}). Upon reaching this limit, a notification ("Tool call count has been exhausted. You can no longer call the tool.") is injected into the context before the final generation resumption, ensuring the agent relies on internal reasoning thereafter.

3.3 Environment Interaction Frameworks

Effective Agent RL involving external tools necessitates robust and scalable environment interaction. Integrating execution environments directly into RL training code often creates dependency issues

Algorithm 1 ZeroTIR Rollout with Spontaneous Code Calls

Require: policy π, prompt P, code env E, call budget N

$C \leftarrow P,\ T \leftarrow \emptyset,\ k \leftarrow 0$
while true do
  $(s, \sigma) \leftarrow \pi.\textsc{Generate}(C, \{\text{EOS}, /boxed\{\}, \text{```python}, \text{```}\})$
  $C, T \leftarrow C + s,\ T + s$
  if $\sigma \in \{\text{EOS}, /boxed\{\}\}$ **then return** T
  end if
  if $\sigma =$ ```python **then**
    $k \leftarrow k + 1$
    if $k > N$ **then return** T
    end if
    $(c, _) \leftarrow \pi.\textsc{Generate}(C, \{\text{```}\})$
    $C, T \leftarrow C + c,\ T + c$
    $r \leftarrow E.\textsc{Exec}(\textsc{Extract}(c))$
    $C, T \leftarrow C + \textsc{Fmt}(r),\ T + \textsc{Tokens}(r)$
  elsereturn T
  end if
end while

and hinders modularity. Our initial approach addressed this by implementing the Python code execution environment as an independent, network-accessible service. This decoupled architecture enhances robustness compared to embedding the interpreter locally, as service failures do not crash the main training process. It also improves maintainability and allows the service to be scaled independently using standard web serving technologies like Flask, Gunicorn for concurrency, Nginx for load balancing, and client-side rate limiting with libraries such as 'aiolimiter' to handle numerous concurrent requests stably. The RL framework interacts with this service via standard HTTP calls.

While the decoupled service provides stability and scalability, synchronous interaction can become a bottleneck. To further improve training throughput, especially for large-scale experiments, we implemented an enhanced interaction framework within OpenRLHF based on asynchronous rollout and pipelining. In this setup, each rollout actor manages an asynchronous queue, which is pre-filled with generated experiences. During training, data retrieval from a queue occurs concurrently with the submission of a new rollout task to an actor. Experience generation proceeds asynchronously while the trainer performs parameter updates, and results are collected afterward to refill the queue. An adaptive mechanism handles potential data shortages in the replay buffer due to filtering by synchronously collecting completed rollouts when needed, preventing excessive data accumulation at the rollout actors. This asynchronous pipeline approach yielded significant speedups in our experiments, proving approximately 1.6 times faster than basic asynchronous rollout and over 4 times faster than the initial synchronous decoupled server interaction, enabling more extensive experimentation, particularly with larger models. For detailed design, please refer to our github repository.

4 Experiments

In this section, we detail the experimental setup designed to evaluate our ZeroTIR approach and validate the existence of Agent RL Scaling Law for spontaneous code execution in mathematical reasoning. We describe the datasets, baselines, implementation details, and present the core results demonstrating the effectiveness of our method and the observed scaling phenomena.

4.1 Experimental Setup

Our experiments primarily utilize the Qwen 2.5 Base 7B/32B model, starting directly from pre-trained weights to align with the ZeroRL philosophy. We implement our ZeroTIR approach using standard community frameworks OpenRLHF and Open-Reasoner-Zero, and evaluate key RL algorithms including PPO and Reinforce++. The training dataset consists of ORZ-57kHu et al. [2025a] and deepmathHe et al. [2025] dataset containing verifiable mathematical problems. We call the model trained in this way ZTRL. Model performance is evaluated on a suite of standard mathematical reasoning benchmarks such as MATH500Hendrycks et al. [2021], AIME24/25Jia, Mathematical Association of America, HMMT Feb. 24/25Harvard–MIT Mathematics Tournament, cmimcCarnegie

Table 1: Performance comparison on key mathematical reasoning benchmarks.

Model	Params	Tool	AIME24	AIME25	MATH500	Avg.	Code Prop.
Qwen2.5 Ins.	7B	✗	13.3%	20.0%	75.8%	36.4%	0.0
Qwen2.5 Ins.	7B	✓	16.7%	0.0%	76.4%	31.0%	0.0
Qwen2.5 Math Ins.	7B	✗	13.3%	6.7%	83.2%	34.4%	0.0
Qwen2.5 Math Ins.	7B	✓	20.0%	26.7%	78.0%	41.6%	**95%**
SimpleRL-Zero	7B	✗	33.3%	6.7%	77.2%	39.1%	0.0
rStar-Math	7B	✗	26.7%	-	78.4%	52.6%	0.0
Eurus-2-PRIME	7B	✗	26.7%	13.3%	79.2%	39.7%	0.0
TORL	7B	✓	43.3%	30.0%	82.2%	51.8%	83%
ZTRL	7B	✓	**46.7%**	**30.0%**	**85.2%**	**54.0%**	90%

Mellon Informatics and Mathematics Competition, olymemathSun et al. [2025a] and so on, which are some of the most difficult mathematical data sets out there.

Key RL hyperparameters include a rollout batch size of 128, with 16 samples generated per prompt. We use 1 policy update step and 12 critic update steps per iteration. Micro-batch sizes for training and forward passes are set to 1. Stability and efficiency techniques, including group-accuracy replay buffer filtering and dynamic stop-token based interaction (detailed in Section 3.2), are employed. The decoupled code execution environment (Section 3.3) handles all tool calls. For initial scaling law validation experiments, the maximum tool calls per trajectory were limited ($N_{max} = 20$) for efficiency.The evaluation metrics include greedy decoding (temperature=0), majority voting, pass@k, and the final performance measured under different top-p sampling settings (temperature=1).

4.2 Comparative Performance Analysis

Only Qwen base, beyond Qwen math. To evaluate the effectiveness of our ZeroTIR approach, denoted ZTRL, we present a comparative analysis in Table 1. This table compares our method, trained on the Qwen 2.5 Base 7B model, against relevant baselines and state-of-the-art modelsZeng et al. [2025], Cui et al. [2025], Guan et al. [2025].

The results clearly demonstrate the significant advantage of our ZeroTIR approach. Our 7B ZTRL model achieves a strong average performance of 52.3% across AIME24, AIME25, and MATH500. Furthermore, ZTRL at 52.3% average significantly surpasses the performance of the Instruct model using prompted TIR, and other ZeroRL methods without tool integration like SimpleRL-Zero at 39.1% and Eurus-2-PRIME at 39.7%. Notably, our ZTRL model, trained from the general Qwen 2.5 Base, also outperforms the specialized Qwen 2.5 Math Instruct model when used with its integrated TIR capability, achieving a 52.3% average compared to 41.6%. We also compare our results with TORL Li et al. [2025], significant concurrent work applying ZeroRL with TIR to mathematical reasoning. It is important to note that the reported TORL results utilized the math-specialized Qwen 2.5 Math Base model for training. As shown in Table 1, our ZTRL approach, despite starting from the general Qwen 2.5 Base model, achieves a higher average score of 54.0% versus TORL's 51.8% on the AIME24, AIME25, and MATH500 benchmarks. This highlights the robustness and effectiveness of our specific ZeroTIR implementation in eliciting strong tool-augmented reasoning capabilities even without a domain-specialized base. The high code usage proportion observed for our model, 90%, is comparable to TORL's 83% and correlates strongly with the achieved performance, consistent with the Agent RL Scaling Law discussed.

4.3 Analysis of hyperparameters

Table 2 confirms a strong monotonic relation between the interaction cap N_{max} and accuracy across all model scales. Raising the cap from zero to four or twenty lifts average scores by as much as fifteen percentage points, though gains taper beyond four calls, underscoring an agent reinforcement-learning scaling law in which additional tool use yields better problem solving. Performance also grows with model size. Under identical hyperparameters the 32B model surpasses the 7B variant, which itself exceeds the 1.5B baseline. The code-usage ratio rises non-linearly, implying that larger models either resolve more tasks without code or employ code more efficiently. At the 7B scale Reinforce++ and PPO reach similar final accuracies, yet Reinforce++ converges roughly three hundred steps sooner, attaining near-optimal performance around step four hundred while PPO needs more than seven hundred. DeepMath training offers a small but consistent edge over Orz-57k, so Reinforce++ with DeepMath was selected for 32B runs to combine efficiency and ceiling.

Table 2: Detailed final performance comparison on selected mathematical reasoning benchmarks.

Method	Params	Algorithm	Dataset	Train Env. Iter.	Eval Env. Iter.	aime25	aime24	hmmt feb. 25	hmmt feb. 24	cmimc	olymemath	math500	avg.	code ratio
ZTRL	1.5B	ppo	orz-57k	0	0	10.0%	3.3%	0.0%	0.0%	3.3%	2.0%	55.8%	10.6%	0.000
ZTRL	1.5B	ppo	orz-57k	2	2	3.3%	3.3%	0.0%	0.0%	3.3%	1.25%	60.6%	10.3%	0.073
ZTRL	1.5B	ppo	orz-57k	4	4	10.0%	20.0%	10.0%	0.0%	10.0%	5.0%	59.4%	16.3%	2.161
ZTRL	1.5B	ppo	orz-57k	20	20	13.3%	13.3%	10.0%	0.0%	13.3%	7.75%	62.6%	17.2%	**4.090**
ZTRL	7B	ppo	orz-57k	0	4	26.7%	13.3%	13.3%	6.7%	10.0%	8.2%	80.6%	22.7%	0.143
ZTRL	7B	ppo	orz-57k	20	20	26.7%	50.0%	10.0%	20.0%	16.7%	13.5%	80.2%	31.0%	3.490
ZTRL	7B	Reinforce++	orz-57k	2	2	26.7%	30.0%	16.7%	13.3%	26.7%	12.3%	82.8%	29.7%	3.686
ZTRL	7B	Reinforce++	deepmath	2	2	16.7%	36.7%	20.0%	10.0%	20.0%	13.5%	81.0%	29.6%	1.710
ZTRL	7B	Reinforce++	deepmath	2	4	16.7%	40.0%	16.7%	16.7%	20.0%	13.2%	80.6%	29.1%	2.417
ZTRL	7B	Reinforce++	deepmath	4	2	26.7%	36.7%	16.7%	23.3%	20.0%	12.7%	81.2%	31.3%	2.257
ZTRL	7B	Reinforce++	deepmath	4	4	26.7%	33.3%	20.0%	23.3%	20.0%	12.5%	82.0%	32.1%	2.470
ORZ	7B	PPO	orz-57k	0	0	10.0%	16.7%	0.0%	6.7%	10.0%	7.0%	82.2%	18.9%	0.000
DeepMath-Zero	7B	/	deepmath	0	0	13.3%	23.3%	13.3%	6.7%	10.0%	7.2%	82.4%	22.3%	0.000
ORZ	32B	PPO	orz-57k	0	0	30.0%	40.0%	20.0%	20.0%	30.0%	20.8%	90.6%	35.9%	0.000
ZTRL	32B	Reinforce++	deepmath	2	2	26.7%	53.3%	20.0%	16.7%	20.0%	16.7%	86.2%	34.2%	1.691
ZTRL	32B	Reinforce++	deepmath	2	4	26.7%	50.0%	16.7%	26.7%	23.3%	19.0%	87.8%	35.7%	1.994
ZTRL	32B	Reinforce++	deepmath	4	2	**33.3%**	**56.7%**	**20%**	**26.7%**	33.3%	17.5%	87.8%	**39.3%**	1.558
ZTRL	32B	Reinforce++	deepmath	4	4	30.0%	46.7%	20.0%	23.3%	**36.7%**	**21.8%**	**89.4%**	38.2%	1.863

Table 4.3 shows that data choice still matters at high capacity. DeepMath delivers the highest Max score, sixty percent on HMMT Feb. 25, whereas Orz-trained models provide stronger majority robustness on CMIMC, fifty-three percent Maj versus thirty-three for DeepMath. Proof-focused curricula sharpen peak reasoning, while heterogeneous contest data stabilises consensus. Decoding entropy drives a clear Max–Maj trade-off. With a four-call budget and DeepMath training, raising top-p from zero point seven to one point zero increases Max by seven points on HMMT Feb. 25 yet lowers Maj and Avg on several sets. Reduced entropy boosts agreement with only modest loss of peak score. Lowering the training interaction cap from four to two seldom harms and sometimes improves evaluation accuracy at cap four. On AIME25 Max rises from fifty to sixty-six percent, indicating that the large model quickly internalises useful code heuristics and that excessive training calls may overfit early exploration.

These findings show that very large models benefit less from extreme training interaction budgets, that dataset diversity and decoding entropy must be tuned jointly to balance peak and robust metrics, and that curriculum choice remains influential even at the 32B scale.

Table 3: Four training configurations (step = 350) across five math-contest datasets. Train/Test = interaction limit; orz = orz-57k; dm = deepmath; Max = max@32; Maj = maj@32; P@1 = pass@1; Avg = avg@32.

	A(orz, 4/4, p=0.7)				**B**(dm, 4/4, p=1.0)				**C**(dm, 2/4, p=1.0)				**D**(dm, 4/4, p=0.7)			
Dataset	Max	Maj	P@1	Avg	Max	Maj	P@1	Avg	Max	Maj	P@1	Avg	Max	Maj	P@1	Avg
AIME25	63%	53%	35%	35%	50%	40%	28%	27%	**66%**	40%	30%	29%	56%	36%	31%	30%
AIME24	**76%**	60%	42%	41%	**76%**	63%	48%	48%	66%	63%	42%	42%	**76%**	50%	43%	43%
HMMT Feb. 25	53%	20%	19%	19%	**60%**	33%	25%	25%	50%	23%	22%	22%	50%	33%	22%	22%
HMMT Feb. 24	**53%**	33%	25%	24%	**53%**	33%	25%	24%	46%	26%	22%	22%	50%	40%	26%	26%
CMIMC	60%	53%	34%	34%	**63%**	33%	30%	30%	**63%**	33%	30%	30%	56%	33%	31%	31%

4.4 Analysis of Training Dynamics

Figure 4 presents key training dynamics, offering insights into the learning process and the Agent RL Scaling Law across different experimental settings. The evolution of code-related metrics is particularly revealing. Code Proportion, representing spontaneous code usage frequency, consistently shows an initial dip followed by a significant increase for TIR-enabled models, indicating that the agent learns the utility of the tool over time after overcoming initial generation challenges. Concurrently, the Code in Correct metric, tracking correct answers involving code, rises sharply in conjunction with the Raw Reward Avg, empirically linking effective, learned tool use directly to task success.

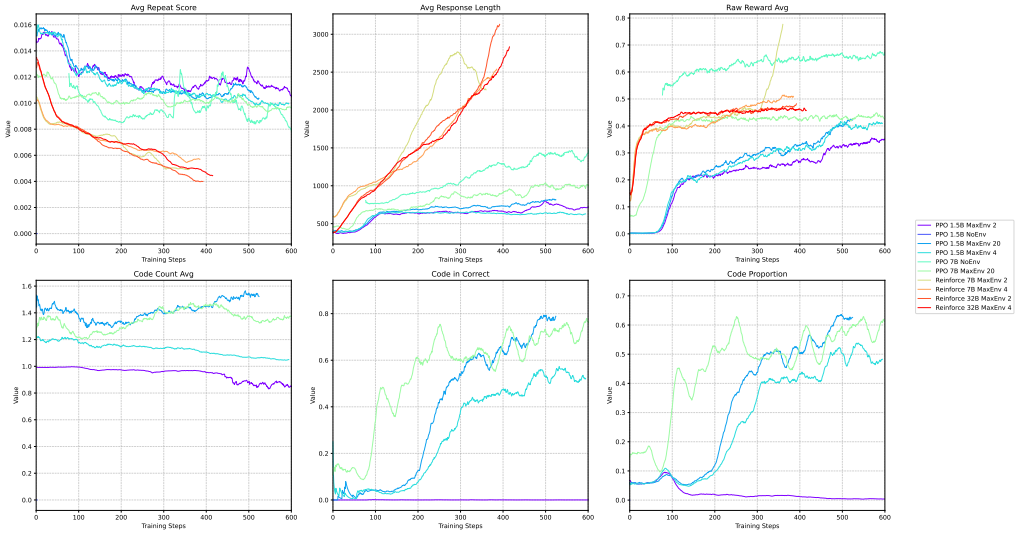


Figure 4: Training dynamics comparing ZeroTIR and ZeroRL in different model and RL algorithm.

Response Length generally increases with training, especially for larger models, correlating with the inclusion of code and outputs, though this trend does not perfectly mirror reward improvements across all settings. The reward curves clearly confirm that enabling tool interaction leads to superior performance compared to non-tool baselines, and further illustrate positive scaling with both increased maximum allowed interactions N_{max} and larger model parameter counts.

Interestingly, while ablations show performance benefits from higher N_{max} values, the Code Count Avg across different settings often stabilizes between 1 and 2 calls per response. This suggests that although a larger interaction budget is available and beneficial, the agents predominantly learn strategies involving few interactions. Combining these observations, we conclude that for base models learning via ZeroTIR, early attempts at multiple interactions might yield poor rewards due to lower code quality, leading the agent to converge towards highly effective single-call strategies. Indeed, our analysis indicates that over 90% of correct responses involving code utilize only a single code execution, with fewer than 10% employing two or more calls, even when permitted.

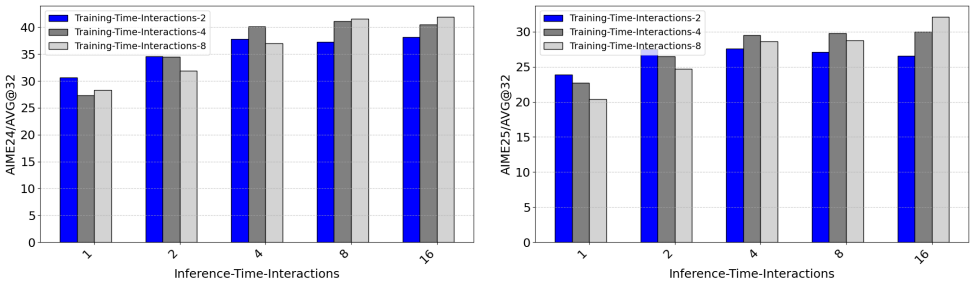
4.5 Joint scaling of training and inference-time interactions

Figure 5 shows the accuracy obtained when we train agents with 2, 4 or 8 interactions per problem and then test them with 1–16 interactions. On every benchmark, allowing more interactions at inference time yields higher scores, but the improvement becomes progressively smaller once the budget reaches 8–16 steps. The curves also reveal a clear interaction between the two budgets: models that were trained with many interactions profit the most from a generous inference budget, yet they can be slightly worse than the low-budget model when only a single step is permitted at test time. In other words, heavy reliance on multi-step reasoning during training can hurt in the one-shot regime, while paying off strongly once at least four inference steps are available.

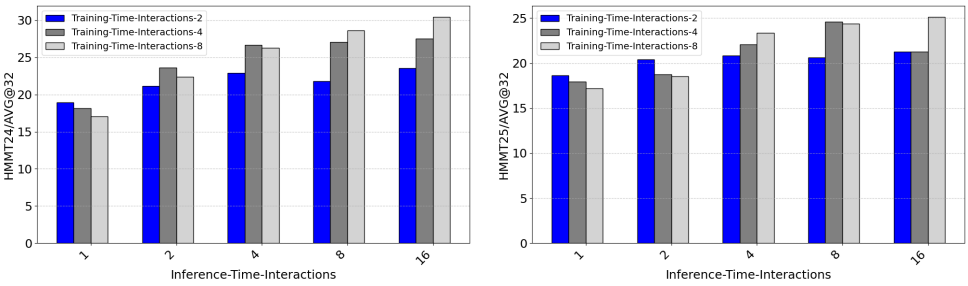
The extent of these effects varies across datasets. On the harder AIME24/25 sets the gap between the 8-interaction model and the 2-interaction model widens to more than ten percentage points when sixteen inference steps are allowed, whereas on HMMT24/25 the corresponding gap is around seven points. Taken together, the results suggest that training and deployment budgets should be aligned: if an application can only afford one or two reasoning steps it is safer to train with the same constraint, but if a larger inference budget is feasible, giving the model ample interaction capacity during training unlocks noticeably better final accuracy.

5 Conclusion

This work investigated the autonomous acquisition of tool use, specifically spontaneous Python code execution for mathematical reasoning, by base Large Language Models trained via outcome-based Reinforcement Learning, ZeroTIR. Our central contribution is the identification and empirical

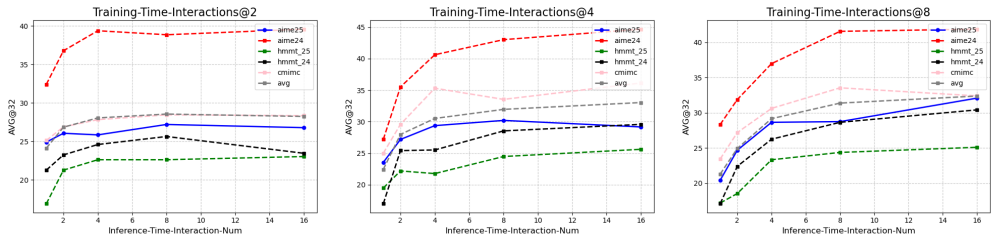


(a) AIME24 (b) AIME25



(c) HMMT24 (d) HMMT25

Figure 5: Training-Time-Interactions and Inference-Time-Interacrtions. We use average@32 to compute the score for AIME24, AIME25, HMMT24 and HMMT25. We use 500-step checkpoint for different training-time-interactions to perform the fair evaluation.



(a) traing-interaction@2 (b) traing-interaction@4 (c) traing-interaction@8

Figure 6: Training time interactions and performance (AVG@32). Increasing the inference interaction number from 2 to 4 to 8 improves all datasets. AIME24 shows the largest and nearly monotonic gains, and AIME25 is second. HMMT 24 and 25 rise steadily. CMIMC peaks around 4 interactions and then slightly declines but remains above 2. The overall average flattens near 8 interactions, so using 8 interactions gives a good balance between accuracy and cost.

characterization of Agent RL Scaling Law governing this learning process. We showed that training progression leads to predictable dynamics in tool usage frequency, code quality, response length, and task accuracy. Key findings include the positive scaling of performance with both model size and the allowed budget for environment interactions, alongside insights into algorithm efficiency where Reinforce++ showed faster convergence than PPO in our 7B experiments, and the deepmath dataset yielded strong results. Crucially, we revealed typical patterns of Tool-Integrated Reasoning usage that emerge during ZeroRL training. Our analysis of training dynamics and interaction counts suggests that while increased interaction potential improves results, models often converge to strategies favoring fewer, high-utility code calls, with a majority of successful tool-using solutions employing only a single execution.

Due to resource limitations, this work primarily focused on empirically demonstrating the existence and qualitative nature of these scaling law. A rigorous quantitative analysis to determine the precise mathematical form of these relationships remains an important direction for future work. Further research should also explore removing constraints on interaction counts entirely and investigating performance in even more complex agentic environments. Overall, our findings advance the un-

derstanding of autonomous tool learning in Agent RL and provide a reproducible framework for studying these critical scaling effects.

References

Carnegie Mellon Informatics and Mathematics Competition. Cmimc math — past problems. https://cmimc.math.cmu.edu/math. Accessed 9 May 2025.

Wenhu Chen, Xueguang Ma, Xinyi Wang, Wenhan Weng, Brian Sadler, Hannaneh Hajishirzi, and Mari Ostendorf. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning. *Transactions on Machine Learning Research (TMLR)*, 2023. arXiv:2211.12588.

Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

Leo Gao, John Schulman, and Jacob Hilton. Scaling Laws for Reward Model Overoptimization. *arXiv preprint arXiv:2210.10760*, 2022a.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-Aided Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022b.

Xinyu Guan, Li Lyna Zhang, Yifei Liu, Ning Shang, Youran Sun, Yi Zhu, Fan Yang, and Mao Yang. rstar-math: Small llms can master math reasoning with self-evolved deep thinking. *arXiv preprint arXiv:2501.04519*, 2025.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Harvard–MIT Mathematics Tournament. Hmmt february problem archive. https://www.hmmt.org/www/tournaments/testing. Accessed 9 May 2025.

Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, et al. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. *arXiv preprint arXiv:2504.11456*, 2025.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. In *NeurIPS Datasets and Benchmarks*, 2021. URL https://arxiv.org/abs/2103.03874.

Zhenyu Hou, Pengfan Du, Yilin Niu, Zhengxiao Du, Aohan Zeng, Xiao Liu, Minlie Huang, Hongning Wang, Jie Tang, and Yuxiao Dong. Does RLHF Scale? Exploring the Impacts from Data, Model, and Method. *arXiv preprint arXiv:2412.06000*, 2024.

Jian Hu. Reinforce++: A simple and efficient approach for aligning large language models. *arXiv preprint arXiv:2501.03262*, 2025.

Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025a.

Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling reinforcement learning on the base model. GitHub repository, 2025b. URL https://github.com/Open-Reasoner-Zero/Open-Reasoner-Zero. URL: https://github.com/Open-Reasoner-Zero/Open-Reasoner-Zero. Version accessed relevant to user's experiments.

Maxwell Jia. Aime 2024 dataset card. https://huggingface.co/datasets/Maxwell-Jia/AIME_2024. Version 2024-03.

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Ö Arık, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

Xuefeng Li, Haoyang Zou, and Pengfei Liu. Torl: Scaling tool-integrated rl. *arXiv preprint arXiv:2503.23383*, 2025.

Mathematical Association of America. American invitational mathematics examination (aime) problems and solutions. https://maa.org/student-programs/amc/. Accessed 9 May 2025.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, William Saunders, and John *et al.* Schulman. WebGPT: Browser-assisted Question-Answering with Human Feedback. *arXiv preprint arXiv:2112.09332*, 2022.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 19(8):198343, 2025.

Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. 'smolagents': a smol library to build great agentic systems. `https://github.com/huggingface/smolagents`, 2025.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*, 2023.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv:2409.19256*, 2024.

Haoxiang Sun, Yingqian Min, Zhipeng Chen, Wayne Xin Zhao, et al. Challenging the boundaries of reasoning: An olympiad-level math benchmark for large language models. *arXiv preprint arXiv:2503.21380*, 2025a.

Sijie Sun, Xiaoxi Jin, Yujia Zhou, Guanting Dong, Yutao Zhang, Jifan Liu, and Zhicheng Dou. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025b.

Haozhe Wang, Long Li, Chao Qu, Weidi Xu, Fengming Zhu, Yi Xin, Wei Chu, and Fangzhen Lin. AutoCode4Math: Learning Autonomous Code Integration for Math LLMs. *arXiv preprint arXiv:2409.14668*, 2024.

Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. Mathcoder: Seamless code integration in ilms for enhanced mathematical reasoning. *arXiv preprint arXiv:2310.03731*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023.

Weihao Zeng, Yuzhen Huang, Wei Liu, Keqing He, Qian Liu, Zejun Ma, and Junxian He. 7b model and 8k examples: Emerging reasoning with reinforcement learning is both effective and efficient. `https://hkust-nlp.notion.site/simplerl-reason`, 2025. Notion Blog.

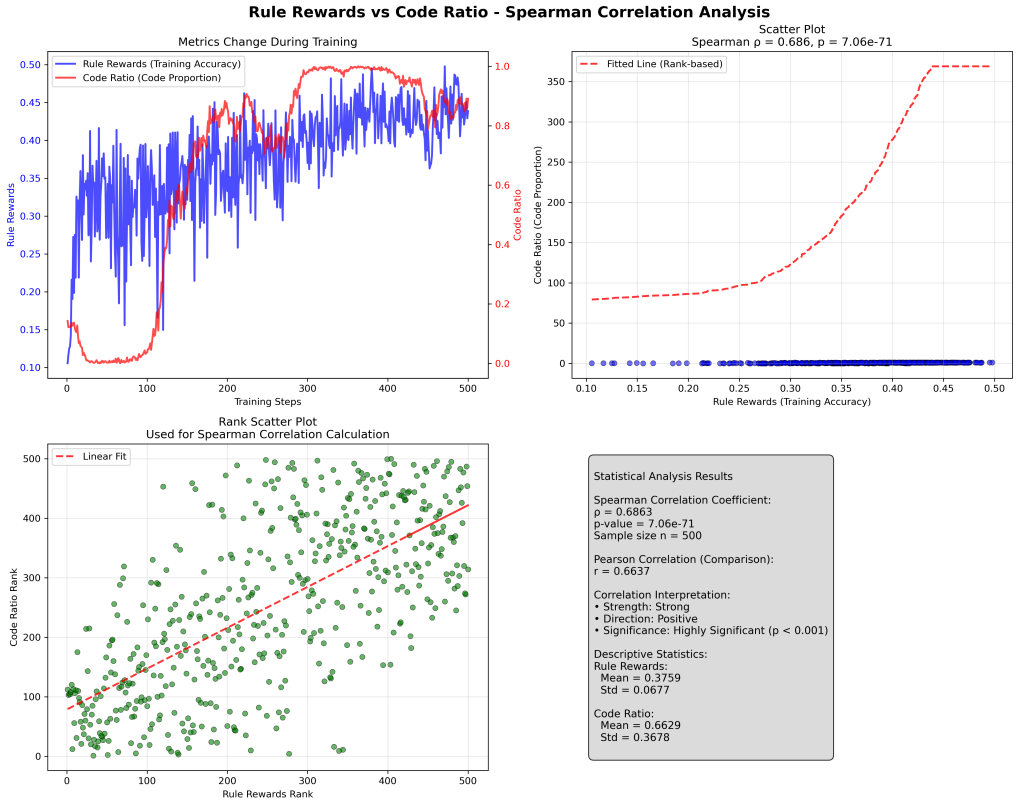


Figure 7: Correlation analysis between the accuracy of the training set and the code ratio during training, which indicating a significant correlation between the two.

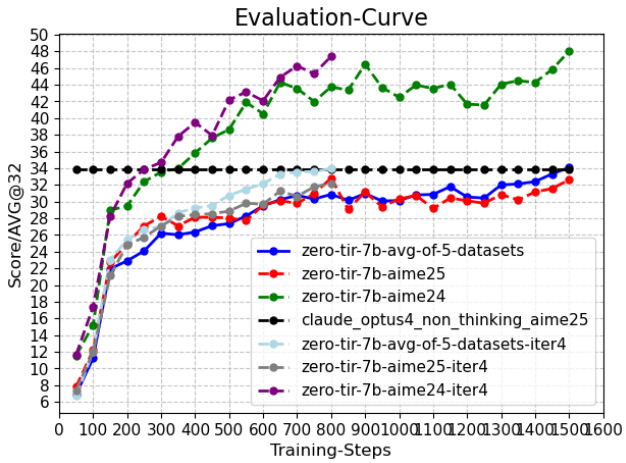


Figure 8: Evaluation curve across training steps for Zero-TIR-7B. The average over five datasets (blue) rises steadily and approaches the Claude Opus 4 non thinking baseline at 33.9% (black). AIME24 (green) surpasses the baseline early and keeps improving into the mid 40s, while AIME25 (red) increases more slowly and ends near the baseline. Iter4 runs (light blue, gray, purple) follow the same pattern with slightly lower scores. Gains taper after about 1300-1500 steps, indicating diminishing returns.